

Question de compréhension

Pourquoi le nom de la classe doit-il correspondre au nom du fichier ?

C'est parce que le compilateur et l'environnement d'exécution Java utilisent le nom du fichier pour localiser et charger les classes. Lorsque la Machine Virtuelle Java (JVM) a besoin de charger une classe appelée Voiture, elle cherche un fichier nommé Voiture.java ou Voiture.class

Que se passe-t-il si vous oubliez le mot-clé private ?

Si on oublie le mot-clé private, les attributs et les fichiers en même package obtiennent l'accès par défaut. Cela signifie que toutes les classes du même package peuvent lire et modifier directement ces attributs.

Pourquoi est-il important d'initialiser les attributs dans le constructeur ?

Il est important d'initialiser les attributs dans le constructeur pour garantir que tout objet créé possède un état valide et cohérent dès sa création. Sans cette initialisation, les attributs prendraient leurs valeurs par défaut (null pour String, 0 pour int, 0.0 pour double), ce qui pourrait provoquer des erreurs comme NullPointerException lors de l'utilisation de l'objet.

Que se passe-t-il si vous ne définissez pas de constructeur par défaut et que vous essayez de créer un objet avec new Voiture() ?

Le programme ne compile pas. Une erreur de compilation apparaît : The constructor Voiture() is undefined. Comme Java fournit automatiquement un constructeur par défaut **uniquement si la classe ne contient AUCUN constructeur explicitement écrit**. Dès qu'on définit un constructeur paramétré, ce constructeur implicite disparaît. Il faut donc **écrire explicitement** le constructeur sans paramètres si on veut continuer à pouvoir faire new Voiture().

Quelle est la différence entre une copie superficielle et une copie profonde ?
(Indice : pensez aux objets de référence comme String).

Copie superficielle (shallow copy) : on copie uniquement les références des attributs de type objet. Les deux objets (original et copie) pointent alors vers les mêmes objets internes. Modifier l'objet interne via l'un affecte l'autre.

Copie profonde (deep copy) : on recrée chaque objet interne référencé, en copiant récursivement son contenu. Les deux objets sont alors totalement indépendants modifier l'un n'affecte en rien l'autre.

Pourquoi ne pas rendre les attributs public au lieu de créer des getters et setters ?

Parce que public donnerait un accès direct et non contrôlé aux données ,n'importe quelle classe pourrait lire ou modifier un attribut sans validation, ce qui rendrait le code fragile et difficile à maintenir. Les getters et setters permettent de contrôler les accès tout en préservant l'encapsulation et la flexibilité interne de la classe.

Que se passe-t-il si vous essayez de définir une année future ?

Le setter setyear() rejette la valeur et affiche un message d'erreur : "Erreur : l'année doit être entre 1886 et 2026.". L'attribut annee n'est PAS modifié il conserve sa valeur précédente. L'objet reste donc dans un état cohérent.

Pourquoi la méthode freiner() vérifie-t-elle si la vitesse devient négative ?

Parce qu'une vitesse négative n'a pas de sens physique une voiture ne peut pas avoir une vitesse inférieure à 0. Si le freinage demandé dépasse la vitesse actuelle, la méthode force la vitesse à 0 (la voiture s'arrête) au lieu de la laisser devenir négative. Cela garantit que l'objet reste toujours dans un état cohérent.

Pourquoi voiture2 == voiture3 retourne-t-il false même si les valeurs sont identiques ?

Parce que l'opérateur == en Java compare les *références* (adresses mémoire) des objets, et non leur *contenu*.

Même si voiture2 et voiture3 ont exactement les mêmes valeurs d'attributs, ce sont deux objets distincts créés par new — ils occupent deux emplacements différents en mémoire. Le constructeur de copie crée un *nouvel objet*, il ne réutilise pas celui de voiture2.

Que se passe-t-il si vous modifiez voiture2 après avoir créé voiture3 ? voiture3 est-il affecté ?

Non, voiture3 n'est pas affecté. Le constructeur de copie crée un nouvel objet indépendant : les valeurs sont copiées, mais les deux objets ne partagent aucune référence. Modifier l'un n'a donc aucun effet sur l'autre.